SPECIFICATION TECHNIQUE

ECF

Vite & Gourmand

Version	Auteur	Motif Version
1.0	N.DENIEUL	Création

Table des matières

1. Contexte 4
2. Stack Technique 4
   2.1 FRONT END 4
   2.2 BACK END 4
   2.3 DEPLOIEMENT 4
3. Configuration Environnement de Travail 4
   3.1 INSTALLATION 4
   3.2 CONFIGURATION 5
4. Modèle de données 6
   4.1 Diagramme de classes 6
   4.2 Base NoSQL (MongoDB) 7
5. Diagramme de cas d'utilisation 8
6. Diagramme de séquence 9
   6.1 Réinitialisation du mot de passe 9
   6.2 Passer une commande 10
7. Sécurité 10
   7.1 Injection SQL 10
   7.2 Faille XSS 10
   7.3 Mots de passe 11
   7.4 Règles de complexité 11
   7.5 Mot de passe oublié 11
   7.6 Contrôle d'accès 11
   7.7 Configuration serveur 12
   7.8 Architecture 13
8. Déploiement AlwaysData 13
   8.1 Fonctionnement 13
   8.2 Base de données 13
   8.3 .env 14
   8.4 Bibliothèques 14
   8.5 Configuration AlwaysData 14

1. Contexte

Dans le cadre de l'obtention du titre professionnel Développeur Web et Web mobile, il est demandé de créer un site web complet selon les exigences à l'obtention du titre.

Les points suivants indiquent les choix et l'organisation techniques du projet.

2. Stack Technique

2.1 FRONT END

HTML 5 : Base de construction (squelette) des pages du site.

CSS : Le CSS permet de gérer l'apparence du site avec utilisation du Framework Bootstrap 5.

JavaScript : Utilisé dans la gestion des filtres de la page « menus » sans rechargement avec AJAX et les graphiques avec Chart.js.

2.2 BACK END

PHP 8.2.12 en local et 8.4 en production, orienté objet sans Framework : langage exécuté côté serveur qui génère les pages HTML. Le projet suit une architecture MVC écrite à la main (routeur index.php, contrôleurs, repositories pour les requêtes SQL, templates pour l'affichage).

Base de données relationnelle : Choix porté sur MySQL qui correspond à mes années d'expérience en SQL, et adaptée à des données liées entre elles (comptes, commandes, menus, plats) par des clés étrangères.

Base de données NoSql : Choix porté sur MongoDB, hébergée sur MongoDB Atlas (imposée par le sujet pour la partie statistiques). On crée une fiche lorsque la commande passe au statut « Terminée ». Celle-ci est reprise dans le volet statistique géré en dehors des données du site.

2.3 DEPLOIEMENT

Déploiement : AlwaysData a été retenu pour son offre gratuite, la prise en charge de PHP, MySQL et de l'extension MongoDB, et son accès SSH qui permet de déployer avec git pull.

3. Configuration Environnement de Travail

3.1 INSTALLATION

- Serveur local

- XAMPP : PHP 8.2.12, MySQL, phpMyAdmin

- Editeurs
- Visual Studio Code
- Notepad++

- Base de données
- MongoDB Atlas avec extension php_mongodb active dans XAMPP
- Interface web MongoDB Atlas : consultation des fiches
- phpMyAdmin

- Dépendances
- Composer (gestion des bibliothèques PHP : MongoDB, Mails, .env)

- Versionning
- Git
- GitHub

- Conception
- Figma
- Draw.io

- Mails
- MailTrap

- Gestion de projet
- Trello

- Tests
- Navigateur + DevTools

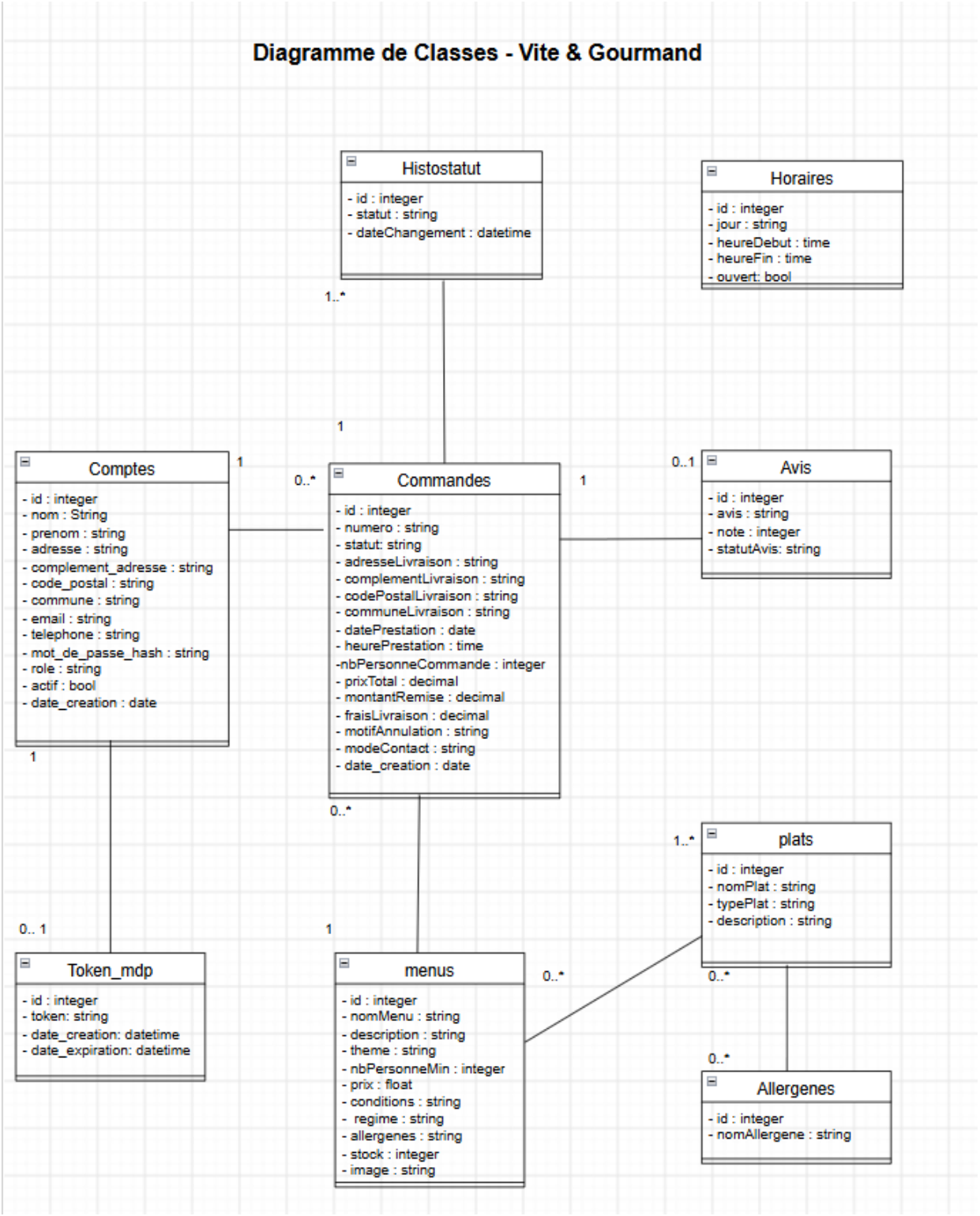
3.2 CONFIGURATION

- .env : Ensemble des variables propres à chaque machine non versionné (déclaré dans .gitignore non transporté car contient les mots de passe) :
- BDD
- Mails
- MongoDB
- APP_URL
- .env.example : Modèle de .env
- Composer install qui permet de récupérer les bibliothèques
- Commande pour lancer en local : php -S localhost:8000 -t public

4. Modèle de données

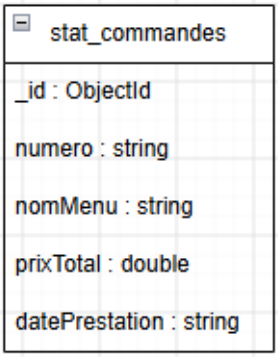
4.1 Diagramme de classes

Le diagramme représente les tables de la base MySQL, leurs attributs et leurs relations.



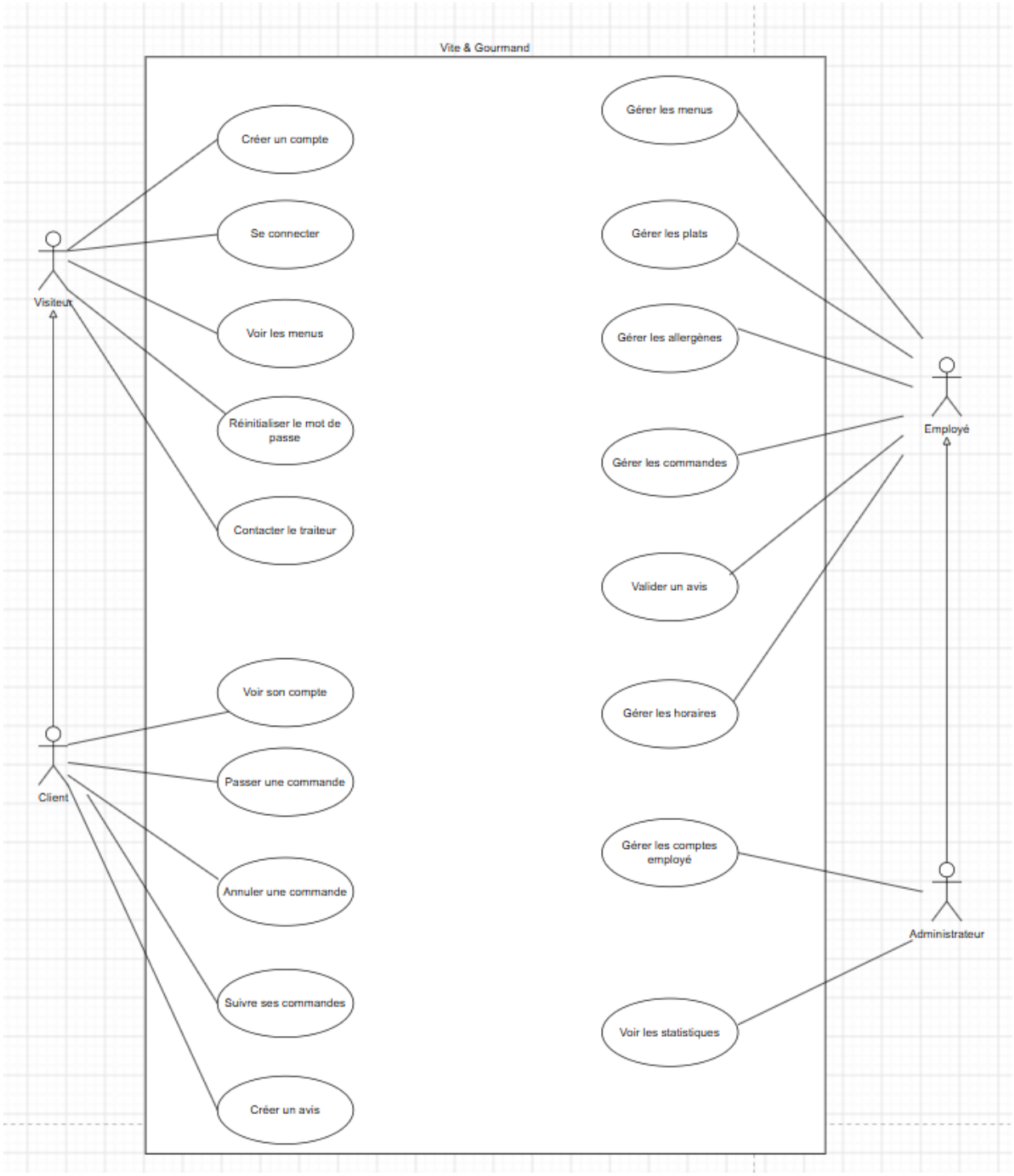
4.2 Base NoSQL (MongoDB)

Fiche stat_commandes MongoDB




```
{
  "_id": {…},
  "numero": "CMD-2026-1044",
  "nomMenu": "Menu Saveurs de saison",
  "prixTotal": 210,
  "datePrestation": "2026-10-14"
}
{
  "_id": {…},
  "numero": "CMD-2026-22",
  "nomMenu": "Menu Délices de la Mer",
  "prixTotal": 226,
  "datePrestation": "2026-10-24"
}
{
  "_id": {…},
  "numero": "CMD-2026-5896",
  "nomMenu": "Menu Evasion Gourmande",
  "prixTotal": 378,
  "datePrestation": "2026-10-27"
}
{
  "_id": {…},
  "numero": "CMD-2026-23",
  "nomMenu": "Menu Délices de la Mer",
  "prixTotal": 508.5,
  "datePrestation": "2026-10-28"
}
```

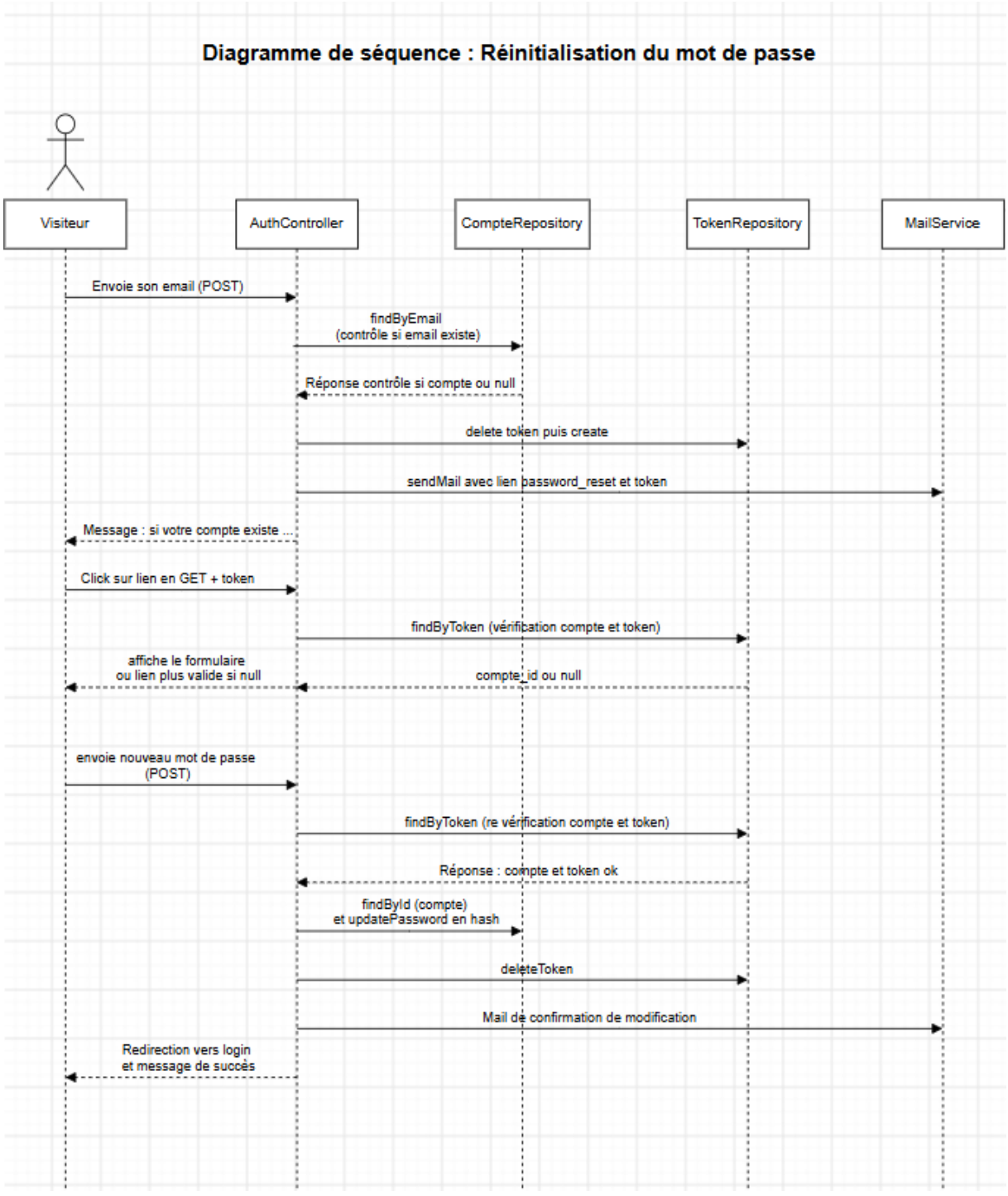
5. Diagramme de cas d’utilisation



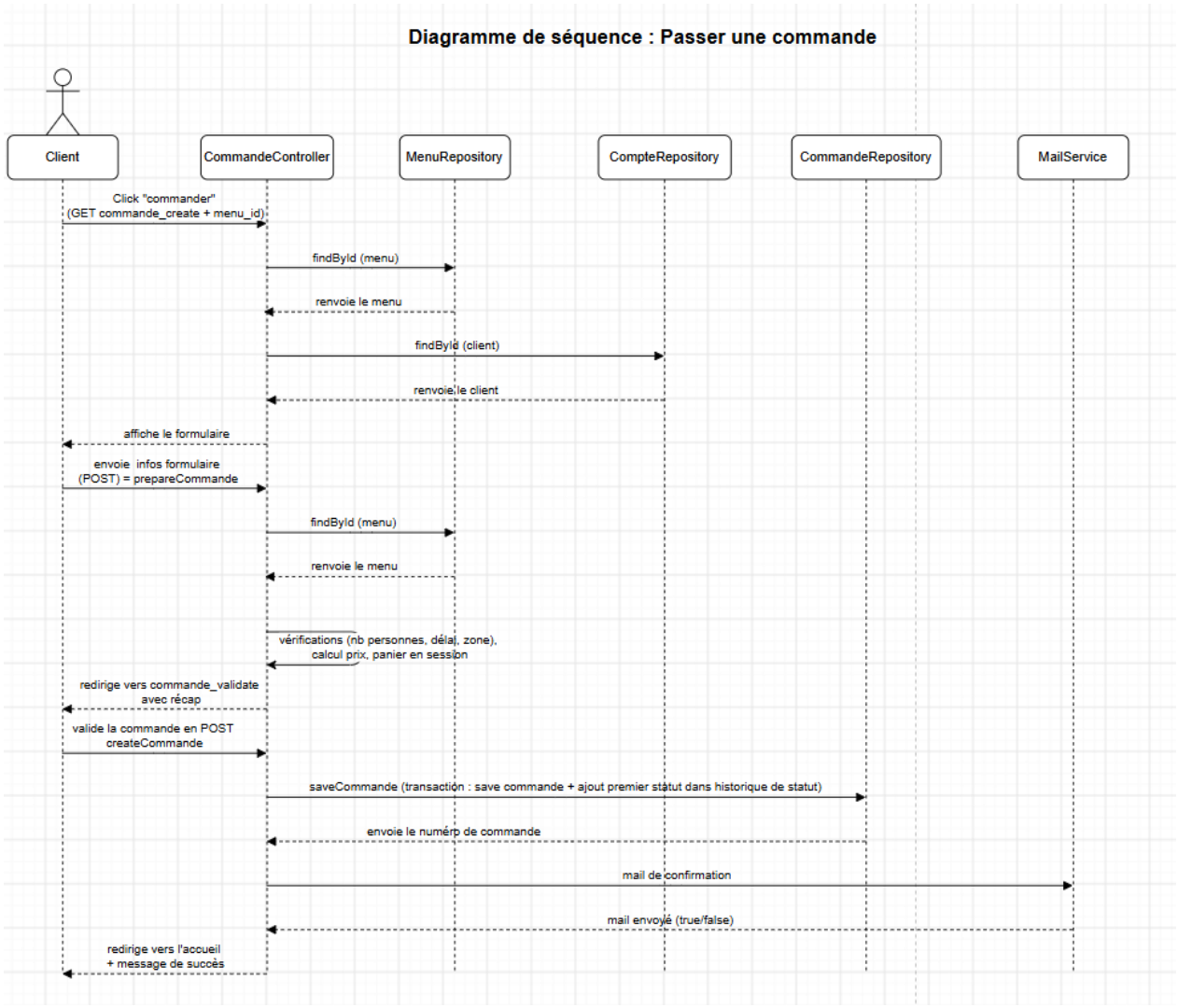
6. Diagramme de séquence

6.1 Réinitialisation du mot de passe

Le token est vérifié à l'affichage du formulaire puis à nouveau à l'envoi et enfin supprimé après utilisation.



6.2 Passer une commande



7. Sécurité

7.1 Injection SQL

- Requêtes préparées PDO avec marqueurs :
  - prepare() envoie la requête à MySQL avec un trou (:email)
  - trou comblé par execute avec la valeur saisie, envoyée séparément ([':email' => $email])
  - Résultat : la saisie ne peut donc jamais modifier la requête.
- Conversion (int) des id reçus en $_GET. Cela permet de transformer tout texte saisi en $_GET au format entier.

7.2 Faille XSS

- Application de htmlspecialchars dans les templates sur les données envoyées de la base afin de transformer les balises HTML en texte et ainsi ne pas réussir à distinguer la balise HTML du texte.
- Utilisation de textContent à la place de innerHTML dans menus_filtres.js. Le texte n'est pas interprété comme du HTML mais comme du texte.

7.3 Mots de passe

- Utilisation de la fonction native de PHP password_hash() avec en paramètre PASSWORD_BCRYPT qui choisit l'algorithme de hachage bcrypt. Le mot de passe est transformé en empreinte irréversible.
- password_verify() : fonction qui permet de vérifier que le mot de passe haché correspond bien au mot de passe fourni (renvoie true ou false).

7.4 Règles de complexité

- Demande de création d'un mot de passe complexe : 10 caractères, 1 majuscule, 1 minuscule, 1 chiffre, 1 caractère spécial.
  preg_match('/[A-Z]/', $password) > doit contenir une majuscule au moins
  && preg_match('/[a-z]/', $password) > doit contenir une minuscule au moins
  && preg_match('/[0-9]/', $password) > doit contenir un chiffre au moins
  && preg_match('/[^a-zA-Z0-9]/', $password) > doit contenir un caractère spécial au moins
  && strlen($password) >= 10 > doit contenir au moins 10 caractères.

  Le contrôle est effectué côté serveur dans register() lors de l'inscription et dans passwordReset() à la création d'un nouveau mot de passe en cas d'oubli.
  Un mot de passe complexe, de niveau fort limite le risque d'être découvert et utilisé.

7.5 Mot de passe oublié

- Utilisation des règles de complexité lors de la création du nouveau mot de passe.
- Protection contre l'énumération de comptes avec la mise en place d'un message neutre sans indication de mail existant. Ainsi une personne malveillante ne peut savoir si un email existe ou non.
- Mise en place de la génération d'un token aléatoire (random_bytes, 64 caractères) avec une durée de validité de 1 heure à partir de sa création. Ce token est vérifié deux fois avec findByToken() à la réception du lien de réinitialisation envoyé par le client et à la réception du formulaire (POST) envoyé par le client.
  A la création du nouveau mot de passe après l'update dans CompteRepository, le token est supprimé. Un nouveau est recréé à chaque nouvelle demande (un token actif par compte).
  Un mail de confirmation de changement est envoyé au client.

7.6 Contrôle d'accès

- Les accès :
  Les accès sont vérifiés via trois fonctions appliquées dans index.php pour la gestion des routes admin, employé, client :
- checkLogin() : vérifie si l'utilisateur est connecté afin d'accéder à l'ensembles des pages de compte client, commande, avis, …
- checkEmploye() : vérifie le rôle de l'utilisateur connecté. Si ce dernier est Employé, il peut accéder à l'ensemble des pages admin hors statistiques et comptes employés.
- checkAdmin() : vérifie le rôle de l'utilisateur. Si le rôle est admin alors il a accès à l'ensemble des pages de l'employé et en plus les statistiques et les comptes employés.

  Un client ne peut visualiser que ses propres informations et ses commandes.

  La route vérifie le rôle de chacun et refuse l'accès si cela ne correspond pas même si l'adresse est tapée directement dans le navigateur.

- Le rôle :
  Par défaut le rôle appliqué lors de la création d'un compte est 'utilisateur'. Seul l'administrateur peut créer un compte 'employé'.
- Actif :
  Si un compte est désactivé, il ne peut plus se connecter.

7.7 Configuration serveur

- .env : fichier contenant toutes les informations de connexions (identifiants, mots de passe, URL) aux bases de données, à l'application web et à MailTrap pour la gestion des mails.

  Ce fichier n'est pas versionné par git car inclus dans le fichier .gitignore.

- .env.example : fichier contenant les exemples de connexions à l'identique de .env sans les informations d'identification et de mot de passe. Il sert d'exemple à la connexion. Le développeur installant le projet doit copier le .env.example et y indiquer ses propres identifiants et mots de passe.

- HTTPS : le site est flaggé HTTPS et non http indiquant que l'ensemble des trajets entre le navigateur et le serveur est protégé (les données sont chiffrées, les données ne peuvent pas être modifiées en chemin, …) comme les informations de connexion.
  Pour le site, HTTPS est fourni par l'hébergeur AlwaysData, l'action « Forcer HTTPS » a été activée. Cela permet de rediriger le site en HTTPS même si on indique http en début d'adresse.

- Erreurs PHP en production :
  La configuration de PHP sur le site de l'hébergeur AlwaysData a été modifiée afin que dans le fichier php.ini, les réglages suivants soient ajoutés :
- display_errors = Off : aucun message technique visible par le visiteur
- log_errors = ON : les erreurs sont enregistrées dans un fichier log sur le serveur de l'hébergeur visible par l'admin ayant accès à l'hébergeur.

  Dans le code, tous les messages d'erreur liés à une action sont neutres et sans informations techniques (suppression d'un menu, erreur de connexion, …).

- MongoDB Atlas :

  MongoDB est une base de données externe qui permet, dans le cadre de Vite & Gourmand, de traiter les données de commandes à des fins de statistiques.
  Comme pour la base MySQL, la connexion à la base s'effectue via les informations (utilisateur et mot de passe fort) renseignées dans le .env (non versionné dans Git).
  Le chemin de connexion entre le site et Atlas est chiffré par le TLS d'Atlas.
  L'hébergeur AlwaysData, en mode gratuit, n'a pas d'IP fixe, il a fallu autoriser toutes les IP. De ce fait, comme il n'y a pas de « gardien » d'IP au préalable, la sécurité d'Atlas repose sur la combinaison identifiants forts + chiffrement TLS. Ainsi sans le bon utilisateur et le bon mot de passe, l'accès est refusé.

7.8 Architecture

Le choix de l'architecture du projet repose sur :

- La sécurité : le fichier .env se trouve à la racine du projet, il n'est pas accessible par un utilisateur car en dehors du dossier /public où se situe index.php. Le fichier index.php est la seule porte d'entrée accessible qui contrôle les accès aux différentes routes en lisant les paramètres et en appelant la bonne méthode dans le controller.
- La séparation des responsabilités : chaque sujet, thème a son fichier repository, controller, template.
    - Le Repository (MODELE) : fichiers qui dialoguent avec la base de données uniquement. Les informations recueillies sont transmises au Controller.
    - Le Controller (CONTROLEUR) : reçoit l'information de la base et du navigateur. Il sert d'intermédiaire entre le site et la base de données. Aucune information du site va directement à la base et inversement. Et il choisit le template à afficher et lui transmet les données.
    - Le Template (VUE) : ensemble des pages qui constitue le squelette des pages du site permettant son affichage. Il ne contient pas de SQL.

8. Déploiement AlwaysData

8.1 Fonctionnement

L'ensemble du code, arborescence (sauf fichiers indiqués dans .gitignore) est développé en local.

Chaque développement est créé en local, rattaché à une branche puis envoyé sur GitHub (un sujet = une branche). Par exemple, tout ce qui concerne les menus est rattaché à la branche feature/menus.

Ces branches de développement sont rattachées à une branche principale : develop qui elle, est rattachée à une branche « finale » : main.

Une fois, le développement terminé, testé, il est fusionné (merge) sur la branche « develop » et envoyé sur GitHub (push).

De nouveaux tests sont effectués sur « develop » puis une fois validés sont fusionnés sur la branche main (git merge puis git push)

Chaque développement validé sur la branche « main » est récupéré avec la commande pull sur le serveur de l'hébergeur AlwaysData (SSH vers le serveur AlwaysData, cd dans le dossier du projet, puis git pull).

8.2 Base de données

Concernant les données, les tables de la base de données, ajoutées pendant le développement, celles-ci ne sont pas transportées par les commandes git. Il convient donc de se connecter à phpMyAdmin depuis le site de l'hébergeur et de lancer les requêtes SQL.

Ces requêtes sont disponibles dans les fichiers schema.sql et donnees.sql.

8.3 .env

Pour mettre à jour le fichier .env (non versionné voir 7.7) en production, il faut accéder au serveur SSH de AlwaysData.

Ensuite taper la commande cd www/vite-et-gourmand puis la commande nano .env qui permet d'accéder au fichier .env. Modifier les informations en se déplaçant avec les touches directionnelles et faire CTRL + O puis ENTREE pour sauvegarder, CTRL + X pour quitter.

8.4 Bibliothèques

Les bibliothèques ne sont pas transportées en production via les commandes git.

Comme le fichier .env, il faut se connecter au serveur SSH de l'hébergeur, de se positionner sur le répertoire racine du projet (cd) et de lancer la commande « composer install » uniquement si une bibliothèque est ajoutée ou mise à jour.

composer.json et composer.lock sont sur GitHub. composer install lit le composer.lock et réinstalle les mêmes versions qu'en local.

8.5 Configuration AlwaysData

Voir 7.7 et 7.8 pour les détails.

- répertoire racine du site = public/ ;
- extension mongodb installée et déclarée dans Environnement > PHP ;
- display_errors / log_errors ;
- Forcer HTTPS.